Exercícios de Python Intermediário

Sugestão: crie uma pasta `exercicios/` e um arquivo por exercício (`ex01.py`, `ex02.py`...). Rode com `python ex01.py` no terminal integrado do VS Code e, se quiser, escreva testes com `pytest`.

1. Funções e estruturas de dados

1. **Contador de palavras:** leia um texto e mostre as 5 palavras mais frequentes (use dict ou collections.Counter).
2. **Anagramas:** dada uma lista de palavras, agrupe as que são anagramas entre si.
3. **Achatar lista:** transforme [[1, 2], [3, [4, 5]], 6] em [1, 2, 3, 4, 5, 6] usando recursão.
4. **Interseção e diferença:** dadas duas listas, mostre elementos comuns, exclusivos de cada uma e a união (use set).
5. **Agrupar por critério:** dada uma lista de dicionários de pessoas (nome, idade, cidade), agrupe por cidade.

2. Comprehensions, lambda e funções de ordem superior

6. **Tabela de quadrados:** gere um dicionário {n: n**2} de 1 a 20, só com números pares, usando dict comprehension.
7. **Ordenação customizada:** ordene uma lista de tuplas (nome, nota) por nota decrescente e, em caso de empate, por nome.
8. **Pipeline funcional:** com map, filter e reduce, calcule a soma dos quadrados dos números ímpares de uma lista.
9. **Decorator de tempo:** crie um decorator @cronometro que mostre quanto tempo uma função levou para executar.
10. **Decorator com argumentos:** crie @repetir(n) que executa a função n vezes.

3. Orientação a objetos

11. **Conta bancária:** classe Conta com depositar, sacar (com validação de saldo) e extrato. Lance exceções personalizadas.
12. **Herança:** crie Animal e subclasses Cachorro e Gato com polimorfismo no método falar().
13. **Dunder methods:** crie uma classe Vetor2D com __add__, __sub__, __mul__, __eq__ e __repr__.
14. **Dataclass:** modele um Produto com @dataclass, compare produtos e ordene por preço.
15. **Sistema de biblioteca:** classes Livro, Usuario e Biblioteca com empréstimo e devolução.

4. Iteradores e geradores

16. **Fibonacci com gerador:** use yield para gerar a sequência sem guardar tudo na memória.
17. **Leitor em blocos:** crie um gerador que lê um arquivo grande linha a linha e filtra as que contêm uma palavra.
18. **Iterador próprio:** implemente uma classe com __iter__ e __next__ que conta regressivamente.

5. Arquivos, JSON e CSV

19. Agenda de contatos: CRUD no terminal salvando os dados em um arquivo .json.

20. Análise de CSV: leia um CSV de vendas (invente um) e calcule total por mês e produto mais vendido.

21. Organizador de pastas: com pathlib, mova arquivos de uma pasta para subpastas por extensão (.jpg, .pdf...).

22. Conversor: converta um CSV para JSON e vice-versa.

6. Tratamento de erros e testes

23. Calculadora robusta: trate ZeroDivisionError, ValueError e entradas inválidas com try/except/else/finally.

24. Validador de CPF: implemente a validação e escreva testes com pytest (casos válidos, inválidos e com pontuação).

25. Context manager: crie um with próprio (via classe ou contextlib) que mede o tempo de um bloco de código.

7. Bibliotecas e projetos maiores

26. Consumo de API: use requests para consultar uma API pública (ex.: ViaCEP ou PokéAPI) e formatar o resultado.

27. Web scraping simples: com requests + BeautifulSoup, extraia títulos de uma página de notícias.

28. Banco de dados: refaça a agenda do exercício 19 usando sqlite3.

29. CLI: crie uma ferramenta de linha de comando com argparse (ex.: contador de linhas/palavras de arquivos).

30. Concorrência: baixe 10 URLs sequencialmente e depois com ThreadPoolExecutor, comparando os tempos.

Desafios finais (projetos)

- **Gerenciador de tarefas** com classes, persistência em JSON e CLI.
- **Jogo da forca ou da velha** com OOP e testes.
- **Mini-analisador de gastos** que lê um CSV de despesas, categoriza e gera um relatório.

Dicas para o VS Code

- Instale a extensão **Python** (Microsoft) e, se quiser, **Pylance**.
- Use Ctrl+Shift+P → *Python: Select Interpreter* para escolher a versão/ambiente virtual.
- Crie um ambiente virtual por projeto: `python -m venv .venv`.
- Use o depurador (F5) com breakpoints para entender o fluxo dos geradores e decorators.